ML-I LAB PROJECT GUIDE & REQUIREMENTS

P34 - Card Fraud Catcher

Catch fraudulent card swipes hidden among normal ones.

Student	PANCHAL JAIMIT - B27 - Div D - 24004501210237
Problem	Detect fraudulent credit-card transactions from Time, Amount and 28 anonymised PCA features (V1-V28).
Dataset	Kaggle: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
File	creditcard.csv (Placed in D:\ML Practical)
Target (Y)	Class (1 fraud / 0 normal) - Highly imbalanced
Type	Binary classification
Model	Random Forest Classifier
Main Metrics	Recall, Precision, F1-score and AUC-ROC for fraud

Deliverables & Objectives

- At the end of this practical you have ONE comprehensive notebook that loads creditcard.csv, cleans it, explores it with charts, predicts 'Class' using Random Forest, compares 4+ settings in a summary table, and predicts 3 new inputs.
- Primary emphasis is placed on Recall and AUC: in fraud detection, missing a fraud transaction (False Negative) is vastly more costly than a false alarm (False Positive).

Project Steps & Implementation Code

1 Create your notebook and load the libraries

- Open Jupyter Notebook or Google Colab and create a new notebook.
- Rename it as RollNo_P34_Name.ipynb (e.g. B27_P34_Panchal_Jaimit.ipynb).
- Add a top Markdown cell with project title, name, roll no, enrollment no, division.
- Run the import block to confirm all dependencies are installed.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, classification_repo
  rt,
   accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, RocCurveDispla
  y)
from sklearn.ensemble import RandomForestClassifier
import warnings; warnings.filterwarnings("ignore")
print("All libraries loaded!")
```

[Note] Write in notebook: Title of project and your personal details.

2 Load your dataset

- Ensure creditcard.csv is in the workspace folder D:\ML Practical.
- Load the dataset using pandas read_csv and check its dimensions.
- Inspect the first 5 records with df.head().

```
df = pd.read_csv("creditcard.csv")
print("Dataset shape:", df.shape) # (284807 rows, 31 columns)
df.head()
```

[Note] Write in notebook: Total rows (284,807) and columns (31), Kaggle source link.

3 Understand the data

- Check every column: examine data types (all numeric float64 except Class int64).
- Verify missing values: creditcard.csv has zero null values in raw state.
- Identify duplicates: dataset contains 1,081 duplicate transactions.
- Compile Data Dictionary: Time, V1-V28 (PCA transformed), Amount, Class.

```
df.info()                # column names, datatypes, non-null counts
df.describe()              # summary statistics for numeric features
print("Missing values per column:\n", df.isnull().sum())
print("Duplicates found:", df.duplicated().sum())
```

[Note] Write in notebook: Data dictionary table + notes on anonymization & PCA features.

4 Prepare the target column (Y)

- Target variable is Class (0 = normal transaction, 1 = fraudulent).
- Extreme class imbalance: 284,315 normal (99.83%) vs only 492 fraud (0.17%).
- A naive model predicting all 0 achieves 99.8% accuracy but catches 0% fraud.

```
TARGET = "Class"
print(df[TARGET].value_counts())
print(df[TARGET].value_counts(normalize=True))
```

[Note] Write in notebook: Class counts and distribution percentages. Note why accuracy alone is deceptive.

5 Clean the data

- Remove the 1,081 exact duplicate transactions to avoid data leakage.
- Impute numeric nulls with column median, categorical with mode if any exist.
- Drop records where the target Class is absent.
- Tree-based algorithms like Random Forest do not require feature scaling.

```
df = df.drop_duplicates()

for col in df.columns:
    if col == TARGET: continue
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())
    else:
        df[col] = df[col].fillna(df[col].mode()[0])
df = df.dropna(subset=[TARGET])

print("Missing values left:", df.isnull().sum().sum())
print("Cleaned shape:", df.shape)
```

[Note] Write in notebook: Summary of cleaned duplicates and why median imputation was configured.

6 Explore the data with charts (EDA)

- Generate count plot to visually demonstrate the severe imbalance.
- Plot feature histograms to observe normal bell-curves vs skewed distributions.
- Create boxplots of Amount vs Class to observe transaction size differences.
- Generate correlation heatmap: PCA features V1-V28 are largely uncorrelated with each other.

```
sns.countplot(x=df[TARGET]); plt.title("Class balance (0: Normal, 1: Fraud)"); plt.show()

num_cols = df.select_dtypes(include="number").columns
df[num_cols].hist(figsize=(14, 10), bins=30); plt.tight_layout(); plt.show()

sns.boxplot(x=df[TARGET], y=df["Amount"]); plt.title("Amount by Class"); plt.show()

plt.figure(figsize=(12, 8))
sns.heatmap(df[num_cols].corr(), cmap="coolwarm"); plt.title("Correlation heat map"); plt.
  show()
```

[Note] Write in notebook: 5 key insights from EDA (e.g. low correlation between PCA features, fraud amount variations).

7 Create new feature, encode & select features

- Create Amount_log = log1p(Amount) to normalize high positive skew in transaction sums.
- Encode labels using LabelEncoder to ensure target is binary 0 and 1.
- SelectKBest (f_classif filter method) keeps top 15 most discriminative features (e.g. V17, V14, V12, V10, V16).
- Dimension reduction speeds up Random Forest training without sacrificing accuracy.

```
# 1) Derived feature
df["Amount_log"] = np.log1p(df["Amount"])

# 2) Feature matrix X and target vector y
X = df.drop(columns=[TARGET])
le_y = LabelEncoder()
y = le_y.fit_transform(df[TARGET])

# 3) One-hot encoding (if any categorical columns exist)
X = pd.get_dummies(X, drop_first=True)

# 4) Feature selection using ANOVA F-score
from sklearn.feature_selection import SelectKBest, f_classif
k = min(15, X.shape[1])
selector = SelectKBest(f_classif, k=k).fit(X, y)
scores = pd.Series(selector.scores_, index=X.columns).sort_values(ascending=False)
X = X[scores.head(k).index]
feature_names = list(X.columns)
print("Selected top features:", feature_names)
```

[Note] Write in notebook: Explanation of Amount_log and the list of 15 selected features.

8 Split into train and test sets

- Partition data into 80% training and 20% testing.
- CRITICAL: Use stratify=y to ensure identical ratio of fraud cases in both sets.
- Random state fixed at 42 for reproducible results across all runs.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print("Train set:", X_train.shape, "Test set:", X_test.shape)
```

[Note] Write in notebook: Sizes of train (80%) and test (20%) sets and verified fraud proportions.

9 Train your model: Baseline Random Forest

- Implement unified evaluation function tracking Accuracy, Precision, Recall, F1, Specificity, and AUC-ROC.
- Display Confusion Matrix and full Classification Report.
- Plot ROC Curve with AUC score indicating separability.

```
results = []

def evaluate(name, model, show=False):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()
    spec = tn / (tn + fp)
    try:
        proba = model.predict_proba(X_test)
        auc = roc_auc_score(y_test, proba[:, 1])
    except Exception:
        auc = None
    row = {"Setting": name,
           "Accuracy": accuracy_score(y_test, y_pred),
           "Precision": precision_score(y_test, y_pred, average="binary", zero_division=0)
  ,
           "Recall": recall_score(y_test, y_pred, average="binary", zero_division=0),
           "F1": f1_score(y_test, y_pred, average="binary", zero_division=0),
           "Specificity": spec, "AUC": auc}
    if show:
        ConfusionMatrixDisplay(cm, display_labels=le_y.classes_).plot(cmap="Blues")
        plt.title(name); plt.show()
        print(classification_report(y_test, y_pred, target_names=[str(c) for c in le_y.cla
  sses_]))
        if auc is not None:
            RocCurveDisplay.from_predictions(y_test, proba[:, 1]); plt.title("ROC - " + na
  me); plt.show()
    print(row)
    return row

# Baseline Random Forest
model = RandomForestClassifier(random_state=42)
results.append(evaluate("Baseline", model, show=True))
```

[Note] Write in notebook: Baseline confusion matrix breakdown (True Negatives, False Positives, False Negatives, True Positives).

10 Hyperparameter Tuning & Feature Importance

- Experiment with tree counts: n_estimators = 50, 100, 200.
- Experiment with tree depth limits: max_depth = None, 5, 10.
- Extract Gini feature importances and plot top 10 features horizontally.
- Explain why ensemble bagging of random trees stabilizes variance compared to single trees.

```
# A) Number of trees (n_estimators)
for n in [50, 100, 200]:
    results.append(evaluate(f"n_estimators={n}", RandomForestClassifier(n_estimators=n, ra
  ndom_state=42)))

# B) Maximum tree depth (max_depth)
for dpt in [None, 5, 10]:
    results.append(evaluate(f"max_depth={dpt}", RandomForestClassifier(n_estimators=100, m
  ax_depth=dpt, random_state=42)))

# C) Feature importance
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train, y_train)
imp = pd.Series(rf.feature_importances_, index=feature_names).sort_values()
imp.tail(10).plot(kind="barh"); plt.title("Feature importance"); plt.show()
print(imp.tail(5))
```

[Note] Write in notebook: Performance differences across settings; top 5 features and their practical interpretation.

11 Compare all settings & pick final model

- Collate results across all evaluated configurations into a pandas DataFrame.
- Plot bar chart comparing Accuracy, Precision, Recall, and F1 across configurations.
- Select optimal model prioritizing Recall (fraud detection) without causing excessive false alarms.
- Retrain final model and display definitive confusion matrix and ROC curve.

```
res = pd.DataFrame(results).round(3)
display(res)

res.set_index("Setting")[["Accuracy", "Precision", "Recall", "F1"]].plot(kind="bar", figsi
  ze=(10, 4))
plt.title("Comparison of settings"); plt.ylim(0, 1); plt.xticks(rotation=45, ha="right");
  plt.show()

best = res.sort_values("Recall", ascending=False).iloc[0]
print("Best setting by Recall:", best["Setting"])

# Final Selected Model
final_model = RandomForestClassifier(n_estimators=200, random_state=42)
evaluate("FINAL", final_model, show=True)
```

[Note] Write in notebook: Comparison table, bar chart, and written justification for chosen hyperparameters.

12 Predict 3 new inputs

- Construct 3 realistic test transactions by perturbing selected feature values.
- Feed them into final_model to get binary predictions and confidence probabilities.
- Verify if predictions conform to expectations (anomalous values trigger fraud flag).

```
# Create 3 synthetic inputs based on real rows with variations
new = X.iloc[[0, 1, 2]].copy()
# Case 1: Extreme V17 anomaly typical of fraud
new.iloc[0, X.columns.get_loc("V17") if "V17" in X.columns else 0] = -15.0
# Case 2: Normal baseline transaction
new.iloc[1] = X.iloc[1]
# Case 3: High amount with negative V14
if "Amount_log" in X.columns: new.iloc[2, X.columns.get_loc("Amount_log")] = 8.5

pred = final_model.predict(new)
pred_prob = final_model.predict_proba(new)[:, 1]

for idx, (p, prob) in enumerate(zip(pred, pred_prob), 1):
    label = "FRAUD" if p == 1 else "NORMAL"
    print(f"Sample {idx}: Predicted {label} (Fraud Probability: {prob:.4f})")
```

[Note] Write in notebook: Values for the 3 samples, predictions, and verification logic.

13 Write report & prepare for viva

- Execute notebook end-to-end: Kernel -> Restart & Run All to ensure zero runtime errors.
- Compile the 4-6 page PDF project report containing charts, tables, and narrative.
- Prepare answers for all 11 lab viva questions.

[Note] Write in notebook: Conclusion detailing model performance, learning outcomes, and real-world deployment considerations.

Viva Voce Questions & Model Answers

- 1. What is bagging? -> Bootstrap Aggregation: training multiple decision trees in parallel on bootstrap samples (random sampling with replacement) and averaging predictions to reduce model variance.
- 2. Why is a forest more stable than one decision tree? -> A single tree suffers from high

variance and easily overfits outliers. A forest averages hundreds of diverse trees, smoothing out errors and stabilizing generalization.

- 3. What does n_estimators change? -> Specifies the number of trees in the forest. More trees improve stability and reduce variance up to a point, after which accuracy plateaus with increased compute time.
- 4. Why does each tree see a random subset of features? -> Feature subsampling decorrelates individual trees. Without it, strong features would dominate every tree root, making all trees similar and defeating the purpose of ensembling.
- 5. Which 5 features were most important and why? -> V17, V14, V12, V10, and V16: PCA transformed features that capture distinct multidimensional behavioral anomalies strongly separating fraud from normal transactions.
- 6. Can Random Forest overfit? -> While resistant due to bagging, it can overfit on very noisy datasets if trees are grown unpruned (max_depth=None) with min_samples_split=1.
- 7. Why is Recall the most important metric in YOUR project? -> Because a False Negative (unflagged fraud transaction) causes direct financial theft, whereas a False Positive (temporary verification check) is easily resolved.
- 8. Precision, Recall & Specificity definitions? -> Precision = TP / (TP + FP) [fraud accuracy]; Recall = TP / (TP + FN) [fraud caught rate]; Specificity = TN / (TN + FP) [normal card correctly passed rate].
- 9. What does AUC = 0.5 and AUC = 1.0 mean? -> AUC = 0.5 indicates random guessing (no discriminative ability); AUC = 1.0 represents perfect classification where all fraud cases rank above all normal cases.
- 10. Why did you use stratify=y in train_test_split? -> Because fraud represents only 0.17% of data. Without stratified sampling, test sets could end up with zero or unrepresentative fraud instances, skewing metrics.
- 11. Which columns did you drop or clean, and why? -> Removed 1,081 duplicate rows to prevent test leakage; dropped ID/unnamed index fields; log-transformed Amount to tame severe right skew.

Submission Checklist & Guidelines

- Working Jupyter Notebook: B27_P34_Panchal_Jaimit.ipynb runs without errors top to bottom.
- Dataset File: creditcard.csv present in project workspace.
- Printed/Generated Project Guide & Report PDF (this document).
- Viva Preparation: Be ready to explain confusion matrix, ROC curve, and hyperparameter impact during the 5-minute practical viva.